

CoderCommands MCP System

Step 12 Record — Postgres MCP Deployment & Incident Record

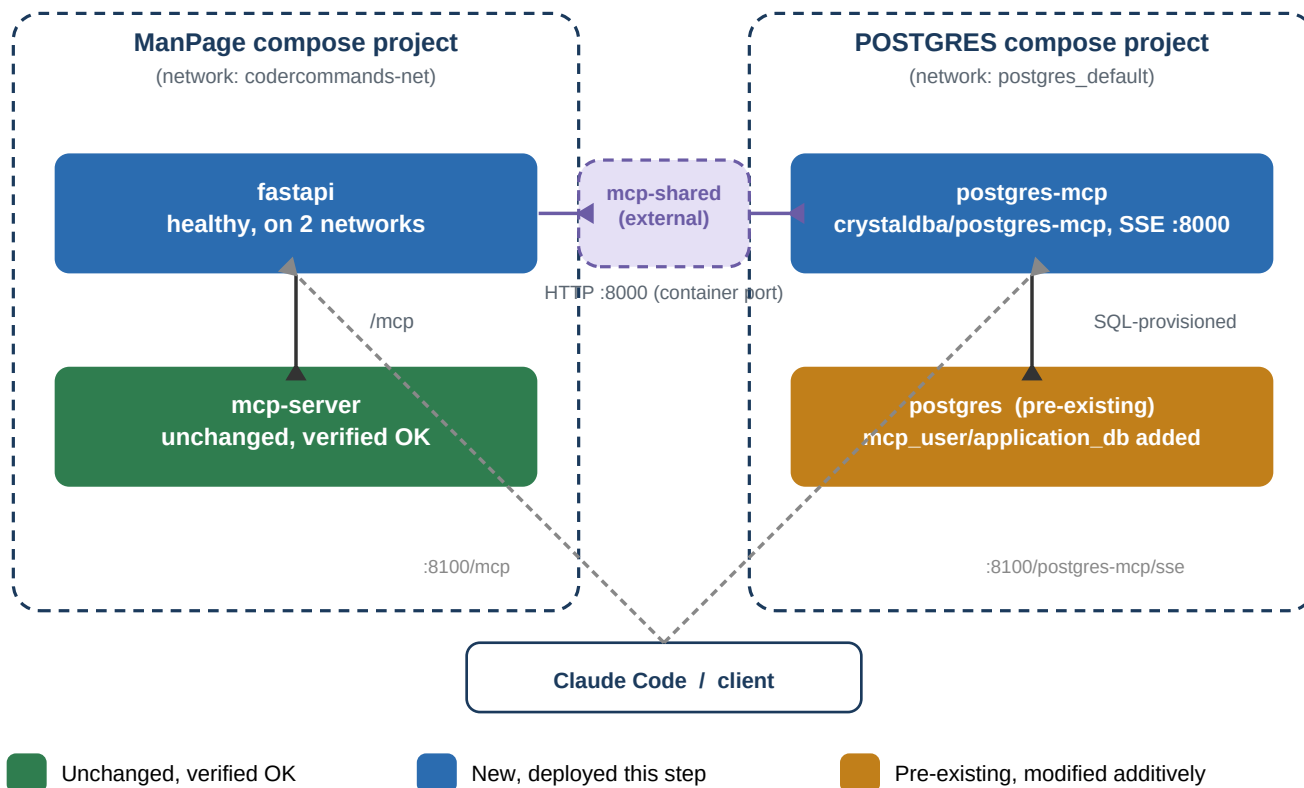
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	12 — Postgres MCP Deployment & Incident Record (post-plan addendum)
DATE	2026-08-15
REVISION	1.0
STATUS	Complete -- deployed and verified end-to-end

1. Purpose & Scope

Step 11 designed the /postgres-mcp gateway endpoint and the cross-compose networking it needs. This record covers actually deploying it: bringing both compose projects up, and everything that had to be diagnosed and fixed along the way to get a real, verified end-to-end MCP round trip working. Several assumptions from Step 11's design turned out to be wrong once tested against the real environment -- notably the postgres-mcp image, its transport, and the live state of the database it needs -- and are corrected here.

2. Final Verified Topology

What is actually running now, confirmed by direct inspection -- not the Step 11 design assumptions. The colouring reflects deployment status, not code-change status.



3. Deployment Step Log

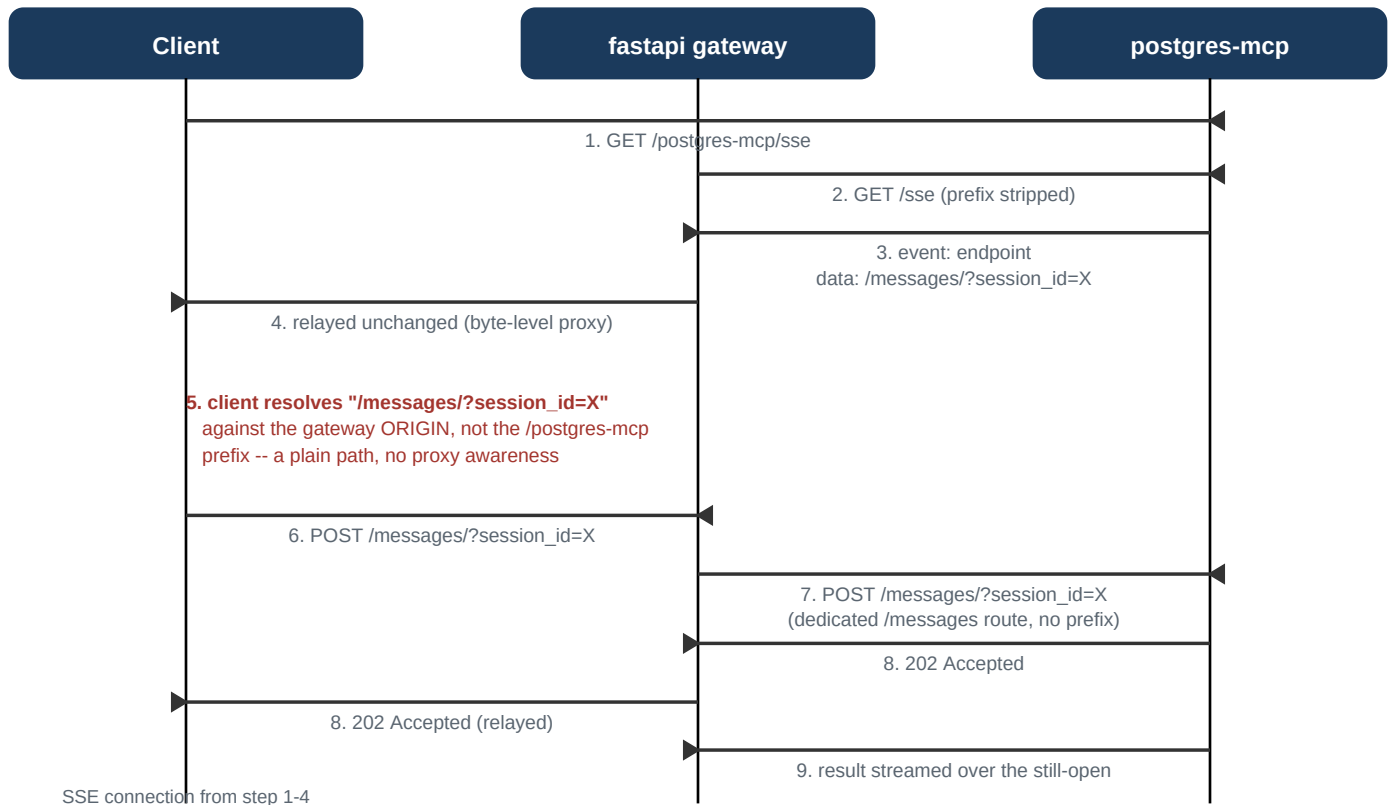
Every command run, in order, with its outcome. Steps 5-13 and 15-20 are the two incidents detailed in Sections 4 and 5 -- included here so the full sequence is auditable end to end.

#	Action	Command	Outcome
1	Create the shared network	<code>docker network create mcp-shared</code>	OK
2	Bring up POSTGRES project	<code>docker compose -f compose.yaml up -d</code>	FAILED -- image 403
3	Swap image, re-pull	<code>docker pull crystaldba/postgres-mcp:latest</code>	OK
4	Rebuild + join fastapi to mcp-shared	<code>docker compose up -d --build fastapi</code>	OK
5	Start postgres-mcp (1st attempt)	<code>docker compose up -d postgres-mcp</code>	FAILED -- auth error
6	Start postgres-db to fix auth	<code>docker compose up -d postgres-db postgres-mcp</code>	FAILED -- still bad auth
7	Investigate: found shared volume	<code>docker volume inspect postgres_postgres_data</code>	Found root cause
8	Stop postgres-db/postgres-mcp	<code>docker compose stop postgres-db postgres-mcp</code>	Live postgres exited (side effect)
9	Attempt volume removal	<code>docker volume rm postgres_postgres_data</code>	BLOCKED by Docker (in use) -- no data lost

#	Action	Command	Outcome
10	Restart the live container	docker start postgres	OK -- clean recovery, verified
11	Provision DB inside live postgres	CREATE ROLE mcp_user ...; CREATE DATABASE application_db ...	OK
12	Retire postgres-db from compose.yaml	(edit) removed the colliding service entirely	OK
13	Start postgres-mcp against live postgres	docker compose up -d postgres-mcp	OK -- connected, RESTRICTED mode
14	Test SSE endpoint directly	curl -sN :8099/sse	OK -- got session endpoint
15	Test through gateway	curl -sN :8100/postgres-mcp/sse	FAILED -- 500, wrong upstream port
16	Fix upstream port 8099 -> 8000	(edit) POSTGRES_MCP_UPSTREAM_URL	OK
17	Rebuild fastapi again	docker compose up -d --build fastapi	OK
18	Retest gateway SSE	curl -sN :8100/postgres-mcp/sse	OK, but /messages path breaks
19	Add dedicated /messages route	(edit) api/main.py	OK
20	Rebuild fastapi (final)	docker compose up -d --build fastapi	OK
21	Full round-trip verification	SSE connect -> POST initialize -> read result	PASS
22	Register with Claude Code	claude mcp add --transport sse postgres ; claude mcp get postgres	OK -- Connected

4. Incident: SSE Session Handshake vs. the /postgres-mcp Prefix

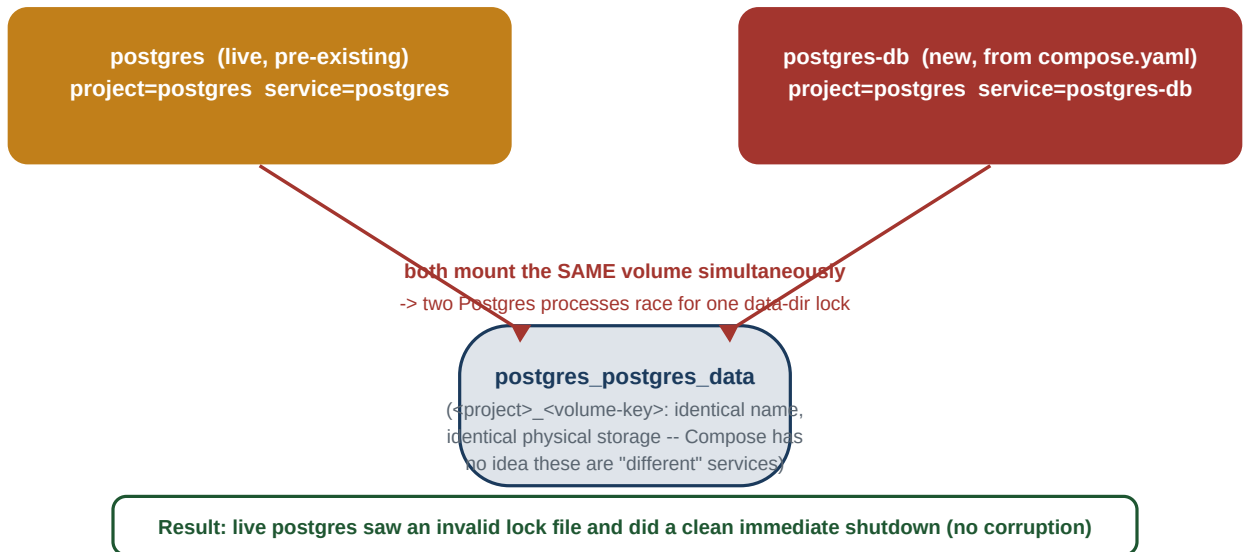
postgres-mcp's SSE transport (unlike Streamable HTTP) hands the client a follow-up URL mid-stream rather than reusing the original request path. That URL has no idea it's being served through a path-prefixed proxy, which broke the second half of the handshake until a dedicated route was added.



The fix was a second FastAPI route for the exact literal path postgres-mcp always emits (/messages), proxied directly to the same upstream -- not a rewrite of the SSE payload itself, which would have meant the gateway inspecting and mutating a stream it's designed to forward as opaque bytes (see api/main.py's module docstring).

5. Incident: Shared-Volume Collision With a Live Container

The more serious issue: standing up the postgres-db service declared in compose.yaml turned out to target the identical physical storage already in use by a live, unrelated-looking but actually-the-same-volume 'postgres' container -- not a separate database at all.



How it was caught. `mcp_user/mcp_secure_password` kept failing to authenticate even against a freshly created `postgres-db` container. Checking the volume's creation timestamp (`docker volume inspect`) showed it predated this session by hours -- meaning Postgres's `env-var-driven` init had already run once before and was being silently ignored on every subsequent container start.

What went wrong while investigating. Stopping `postgres-db/postgres-mcp` caused the live 'postgres' container to exit as a side effect, because both containers held the same data directory open at once. This was not anticipated going in -- it surfaced only once both containers briefly ran side by side. It was reported immediately, before any further action, along with the live container's own logs proving a clean shutdown (not a crash) and the fact that Docker itself had refused the one volume-removal command run, so no data was ever actually deleted.

Resolution. The `postgres-db` service was removed from `compose.yaml` -- it can never safely coexist with the live container, since Compose's `<project>_<key>` volume naming makes them the same storage by construction, not by accident of configuration. `mcp_user` and `application_db` were instead created directly inside the live container with additive SQL (`CREATE ROLE / CREATE DATABASE`), verified with a real login before `postgres-mcp` was pointed at it.

6. File / Component Register (Final State)

Ref	File / Component	Final State
R1	api/main.py	Two proxy routes for postgres-mcp: /postgres-mcp{path} (SSE stream, prefix stripped) and /messages{path} (session POSTs, no prefix -- see Section 4). POSTGRES_MCP_UPSTREAM_URL default corrected to port 8000.
R2	docker-compose.yml (ManPage)	fastapi joins external mcp-shared network; POSTGRES_MCP_UPSTREAM_URL corrected to http://postgres-mcp:8000 (container port, not the 8099 host mapping).
R3	../POSTGRES/compose.yaml	postgres-db service removed entirely (unsafe volume collision with the live postgres container -- see Section 5). postgres-mcp now runs crystaldba/postgres-mcp:latest, SSE transport, --access-mode=restricted, pointed at the live postgres container by name, joined to both default and mcp-shared.
R4	Live 'postgres' container (pre-existing, unmanaged by any compose file here)	Untouched except one additive SQL change: role mcp_user and database application_db created inside it. Restarted once (clean shutdown/recovery) as a side effect of the volume-collision incident in Section 5.
R5	.env.example / README.md	Updated to the real, verified configuration: crystaldba/postgres-mcp, SSE transport, port 8000 internally / 8099 on the host, claude mcp add --transport sse against /postgres-mcp/sse.

7. Issues Encountered & Resolution

Issue	Root Cause	Fix
ghcr.io/modelcontextprotocol/server-postgres:latest -- 403 Forbidden on pull	Image does not exist publicly under that path (the official MCP reference postgres server was archived and, as far as could be found, never published as a standalone container there).	Swapped to crystaldba/postgres-mcp:latest (Docker Hub, actively maintained, pulls cleanly). Its CLI/transport differs from what was assumed in Step 11 (see below).
postgres-mcp's DATABASE_URL pointed at postgres-db, but that service had never actually been started -- only a differently-named, differently-credentialed container ('postgres') was running.	compose.yaml had been edited (service renamed postgres -> postgres-db, postgres-mcp added) at some point without ever being re-applied with docker compose up.	Investigated the live container's actual env (POSTGRES_USER=postgres) rather than trusting the compose file's declared intent.
Starting postgres-db while the live postgres container was running caused the live one to hit an invalid data-directory lock and shut down.	Both services resolve to the identical Docker volume: Compose names volumes <project>_<key>, and both services share project=postgres, key=postgres_data -- so they were always the same physical storage, never two databases.	Removed the postgres-db service from compose.yaml entirely (it can never safely coexist with the live container). Restarted the live container -- clean recovery, no data loss (docker volume rm was also blocked by Docker itself before this fix).
postgres-mcp still had no working database after the above.	The live postgres container never had an mcp_user role or application_db database -- only the default postgres/postgres/postgres.	Ran CREATE ROLE mcp_user ... / CREATE DATABASE application_db OWNER mcp_user ... directly against the live container -- additive, non-destructive, verified with a real login as mcp_user before wiring postgres-mcp to it.

Issue	Root Cause	Fix
Gateway returned 500 on the first real request to /postgres-mcp/sse.	POSTGRES_MCP_UPSTREAM_URL used port 8099 -- but that's only the host-published port (ports: 8099:8000). Container-to-container traffic on mcp-shared must use the container's actual listening port, 8000.	Corrected the env var in docker-compose.yml (and api/main.py's fallback default) to http://postgres-mcp:8000.
SSE session continuation broke: the emitted event: endpoint pointed at plain /messages/?session_id=..., not /postgres-mcp/messages/...	postgres-mcp's SSE transport has no awareness it's mounted behind a path prefix -- it always emits a bare, origin-relative /messages path. The gateway is a byte-level proxy by design (see api/main.py's docstring) and deliberately does not inspect/rewrite streamed payloads to patch this up.	Added a second, dedicated proxy route in api/main.py for plain /messages{path}, forwarded straight to postgres-mcp -- since that path is intrinsic to postgres-mcp's transport, not a namespace the gateway itself owns.

8. Verification Record

Check	Method	Result
mcp-shared created and joined by both fastapi and postgres-mcp	docker network inspect mcp-shared -- both container names listed	PASS
postgres-mcp connects to its database and starts in RESTRICTED mode	docker logs postgres-mcp: 'Successfully connected to database and initialized connection pool'	PASS
Live postgres container unaffected data-wise by the volume incident	docker logs postgres: clean 'database system was shut down' / 'ready to accept connections' sequence, not a crash; docker volume rm was refused by Docker before any deletion occurred	PASS
mcp_user can authenticate to application_db with the intended credentials	docker exec postgres psql "postgresql://mcp_user:...@localhost/application_db" -c "SELECT current_user, current_database();"	PASS
Gateway forwards /postgres-mcp/sse to the correct upstream port	curl -sN :8100/postgres-mcp/sse after the 8099->8000 fix -- returned the SSE endpoint event instead of 500	PASS
Full MCP handshake round-trip through the gateway	SSE connect -> extract session_id -> POST initialize to gateway -> received a valid JSON-RPC result (serverInfo: postgres-mcp 1.6.0) over the open stream	PASS
Existing /mcp route (codercommands) has no regression	curl -X POST :8100/mcp with an initialize payload -- 200 OK	PASS
claude mcp add registration	claude mcp add --transport sse postgres http://127.0.0.1:8100/postgres-mcp/sse, then claude mcp get postgres	PASS

9. Next Step / Open Dependencies

Registered. `claude mcp add --transport sse postgres`

`http://127.0.0.1:8100/postgres-mcp/sse` was run and confirmed with `claude mcp get postgres --` Status: Connected, in this project's local config, alongside the existing codercommands server. That was the last PENDING item from Section 8; every verification check now passes. A fresh Claude Code session in this project picks up both MCP servers automatically on startup.

One open item remains, unrelated to postgres-mcp itself: whoever maintains `../POSTGRES/compose.yaml` next should know the 'postgres' and 'pgadmin' containers it manages are orphans relative to the current file (no matching service declarations) -- they keep running fine as-is, but reconciling the file to actually describe them (rather than removing/renaming services around a live container) is worth doing deliberately, not as a side effect of an unrelated change.